



There are 2 stages where error may happen in a program

- During compilation -> Syntax Error
- During execution -> Exceptions

Syntax Error

- Something in the program is not written according to the program grammar.
- Error is raised by the interpreter/compiler
- You can solve it by rectifying the program

```
In [1]: # Examples of syntax error
print 'hello world'
```

```
Cell In[1], line 2
    print 'hello world'
    ^
```

SyntaxError: Missing parentheses in call to 'print'. Did you mean print(...)?

Other examples of syntax error

- Leaving symbols like colon, brackets
- Misspelling a keyword
- Incorrect indentation
- empty if/else/loops/class/functions

```
In [2]: a = 5
if a==3
    print('hello')
```

```
Cell In[2], line 2
    if a==3
    ^
```

SyntaxError: expected ':'

```
In [3]: a = 5
iff a==3:
    print('hello')
```

```
Cell In[3], line 2
    iff a==3:
    ^
```

SyntaxError: invalid syntax

```
In [5]: a = 5
if a==3:
    print('hello')
```

```
Cell In[5], line 3
    print('hello')
    ^
```

IndentationError: expected an indented block after 'if' statement on line 2

```
In [ ]: # IndexError
        # The IndexError is thrown when trying to access an item at an invalid index.
        L = [1,2,3]
        L[100]
```

```
-----
IndexError                                Traceback (most recent call last)
<ipython-input-71-c90668d2b194> in <module>
      2 # The IndexError is thrown when trying to access an item at an invalid
index.
      3 L = [1,2,3]
----> 4 L[100]

IndexError: list index out of range
```

```
In [ ]: # ModuleNotFoundError
        # The ModuleNotFoundError is thrown when a module could not be found.
        import mathi
        math.floor(5.3)
```

```
-----
ModuleNotFoundError                      Traceback (most recent call last)
<ipython-input-73-cbdaf00191df> in <module>
      1 # ModuleNotFoundError
      2 # The ModuleNotFoundError is thrown when a module could not be found.
----> 3 import mathi
      4 math.floor(5.3)

ModuleNotFoundError: No module named 'mathi'
```

NOTE: If your import is failing due to a missing package, you can manually install dependencies using either `!pip` or `!apt`.

To view examples of installing some common dependencies, click the "Open Examples" button below.

```
In [6]: # KeyError
        # The KeyError is thrown when a key is not found

        d = {'name': 'nitish'}
        d['age']
```

```
-----
KeyError                                Traceback (most recent call last)
Cell In[6], line 5
      1 # KeyError
      2 # The KeyError is thrown when a key is not found
      4 d = {'name': 'nitish'}
----> 5 d['age']

KeyError: 'age'
```

```
In [7]: # TypeError
# The TypeError is thrown when an operation or function is applied to an object
1 + 'a'
```

```
-----
TypeError                                Traceback (most recent call last)
Cell In[7], line 3
      1 # TypeError
      2 # The TypeError is thrown when an operation or function is applied to an object of an inappropriate type.
----> 3 1 + 'a'

TypeError: unsupported operand type(s) for +: 'int' and 'str'
```

```
In [8]: # ValueError
# The ValueError is thrown when a function's argument is of an inappropriate type
int('a')
```

```
-----
ValueError                                Traceback (most recent call last)
Cell In[8], line 3
      1 # ValueError
      2 # The ValueError is thrown when a function's argument is of an inappropriate type.
----> 3 int('a')

ValueError: invalid literal for int() with base 10: 'a'
```

```
In [9]: # NameError
# The NameError is thrown when an object could not be found.
print(k)
```

```
-----
NameError                                Traceback (most recent call last)
Cell In[9], line 3
      1 # NameError
      2 # The NameError is thrown when an object could not be found.
----> 3 print(k)

NameError: name 'k' is not defined
```

```
In [10]: # AttributeError
L = [1,2,3]
L.upper()
```

```
# Stacktrace
```

```
-----  
AttributeError                                Traceback (most recent call last)  
Cell In[10], line 3  
      1 # AttributeError  
      2 L = [1,2,3]  
----> 3 L.upper()  
  
AttributeError: 'list' object has no attribute 'upper'
```

Exceptions

If things go wrong during the execution of the program(runtime). It generally happens when something unforeseen has happened.

- Exceptions are raised by python runtime
- You have to take care of it on the fly

Examples

- Memory overflow
- Divide by 0 -> logical error
- Database error

```
In [40]: # Why is it important to handle exceptions  
        ## userExperience : dont let user see the stacktrace so technical details  
        ## debugging : log the error and stacktrace to debug the issue  
        ## security : dont let user see the stacktrace as it may contain sensitive  
  
        # how to handle exceptions  
        # -> Try except block
```

```
In [41]: # let's create a file  
with open('sample.txt','w') as f:  
    f.write('hello world \nhow are you \nwelcome to exception handling in python')
```

```
In [42]: # try catch demo  
try:  
    with open('sample1.txt','r') as f:  
        print(f.read())  
except:  
    print('sorry file not found')
```

sorry file not found

```
In [43]: # catching specific exception  
try:
```

```

m=5
f = open('sample1.txt','r')
print(f.read())
print(m)
print(5/2)
L = [1,2,3]
L[100]
except FileNotFoundError:
    print('file not found')
except NameError:
    print('variable not defined')
except ZeroDivisionError:
    print("can't divide by 0")
except Exception as e:
    print(e)

```

file not found

```

In [44]: # else : runs if there is no exception
try:
    f = open('sample1.txt','r')
except FileNotFoundError:
    print('file nai mili')
except Exception:
    print('kuch to lafda hai')
else:
    print(f.read())

```

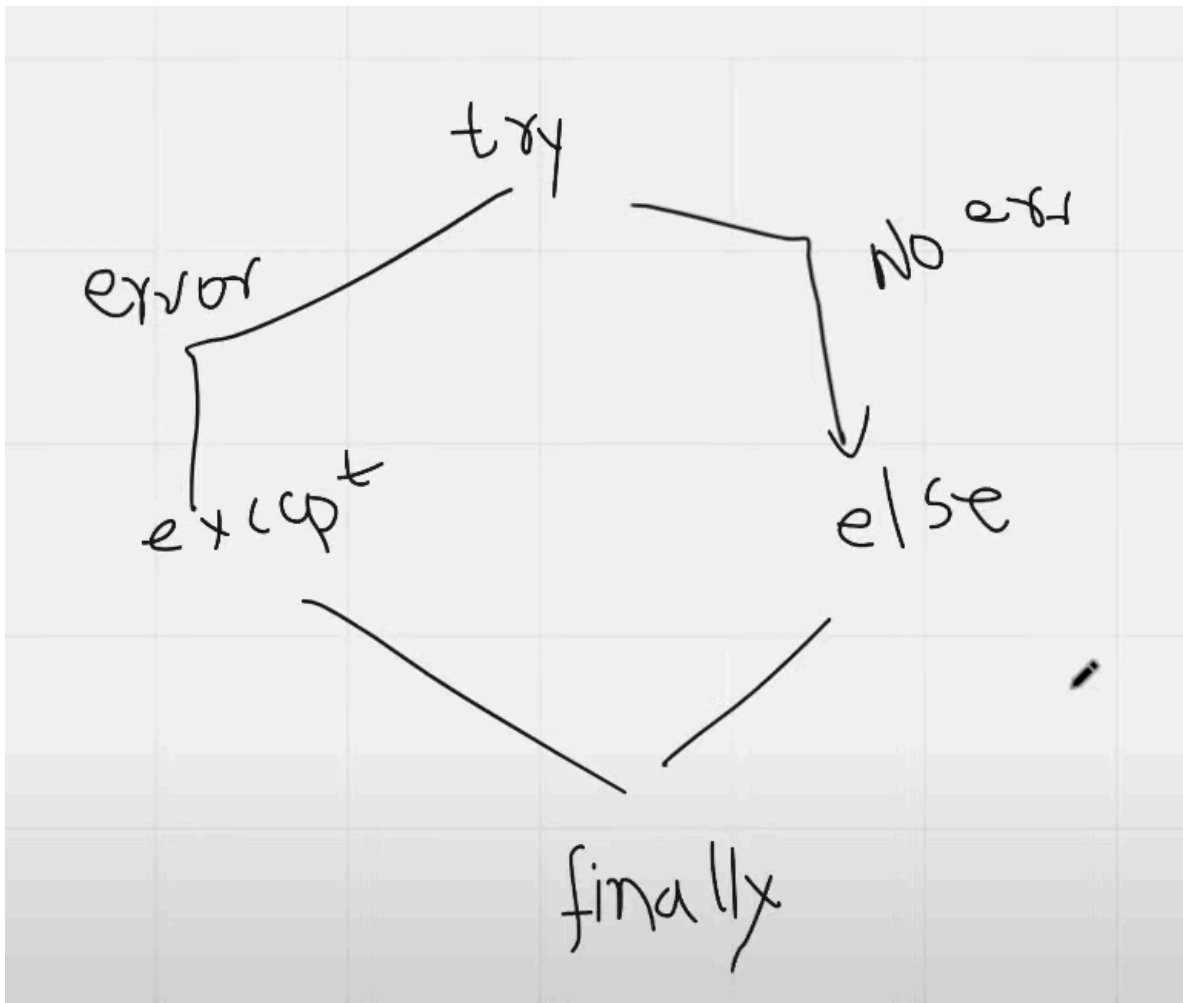
file nai mili

```

In [45]: # finally
# else
try:
    f = open('sample1.txt','r')
except FileNotFoundError:
    print('file nai mili')
except Exception:
    print('kuch to lafda hai')
else:
    print(f.read())
finally:
    print('ye to print hoga hi')

```

file nai mili
ye to print hoga hi



```
In [46]: # raise Exception
# In Python programming, exceptions are raised when errors occur at runtime.
# We can also manually raise exceptions using the raise keyword.

# We can optionally pass values to the exception to clarify why that exception
```

```
In [48]: raise ZeroDivisionError('aise hi try kar raha hu')
# Java
# try -> try
# except -> catch
# raise -> throw
```

```
-----
ZeroDivisionError                                Traceback (most recent call last)
Cell In[48], line 1
----> 1 raise ZeroDivisionError('aise hi try kar raha hu')

ZeroDivisionError: aise hi try kar raha hu
```

```
In [49]: class Bank:

    def __init__(self, balance):
        self.balance = balance
```

```

def withdraw(self,amount):
    if amount < 0:
        raise Exception('amount cannot be -ve')
    if self.balance < amount:
        raise Exception('paise nai hai tere paas')
    self.balance = self.balance - amount

obj = Bank(10000)
try:
    obj.withdraw(15000)
except Exception as e:
    print(e)
else:
    print(obj.balance)

```

paise nai hai tere paas

In []: *## custom exception : python allows us to create our own custom exceptions by*
Advantages of custom exceptions : application specific exceptions can provi

```

class MyException(Exception):
    def __init__(self,message):
        print(message)

class Bank:

    def __init__(self,balance):
        self.balance = balance

    def withdraw(self,amount):
        if amount < 0:
            raise MyException('amount cannot be -ve')
        if self.balance < amount:
            raise MyException('paise nai hai tere paas')
        self.balance = self.balance - amount

obj = Bank(10000)
try:
    obj.withdraw(5000)
except MyException as e:
    pass
else:
    print(obj.balance)

```

5000

In []: *# creating custom exceptions*
exception hierarchy in python

In []: *# simple example*

```
In [50]: class SecurityError(Exception):

    def __init__(self,message):
        print(message)

    def logout(self):
        print('logout')

class Google:

    def __init__(self,name,email,password,device):
        self.name = name
        self.email = email
        self.password = password
        self.device = device

    def login(self,email,password,device):
        if device != self.device:
            raise SecurityError('bhai teri to lag gayi')
        if email == self.email and password == self.password:
            print('welcome')
        else:
            print('login error')

obj = Google('nitish','nitish@gmail.com','1234','android')

try:
    obj.login('nitish@gmail.com','1234','windows')
except SecurityError as e:
    e.logout()
else:
    print(obj.name)
finally:
    print('database connection closed')
```

```
bhai teri to lag gayi
logout
database connection closed
```